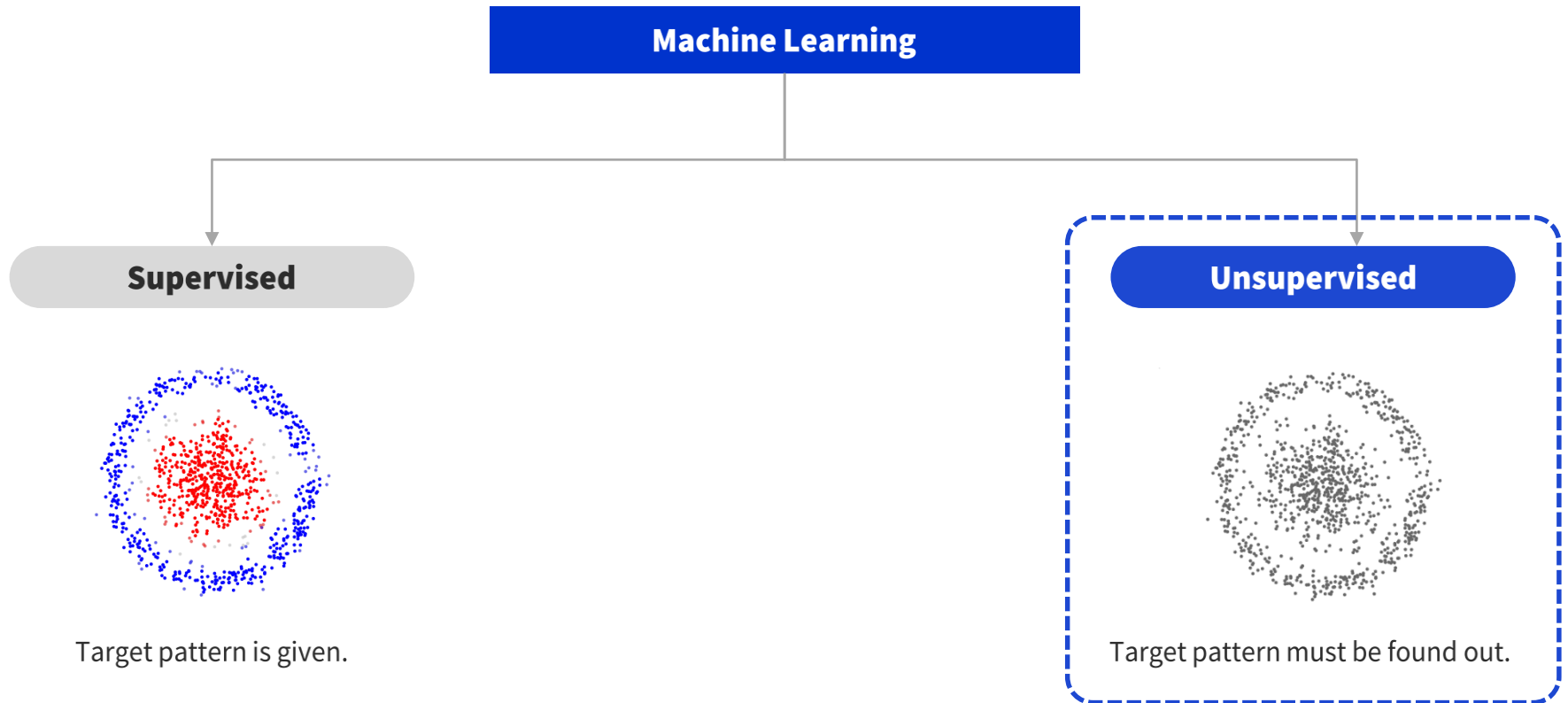


K-MEANS CLUSTERING

thanh.nguyen@vlu.edu.vn

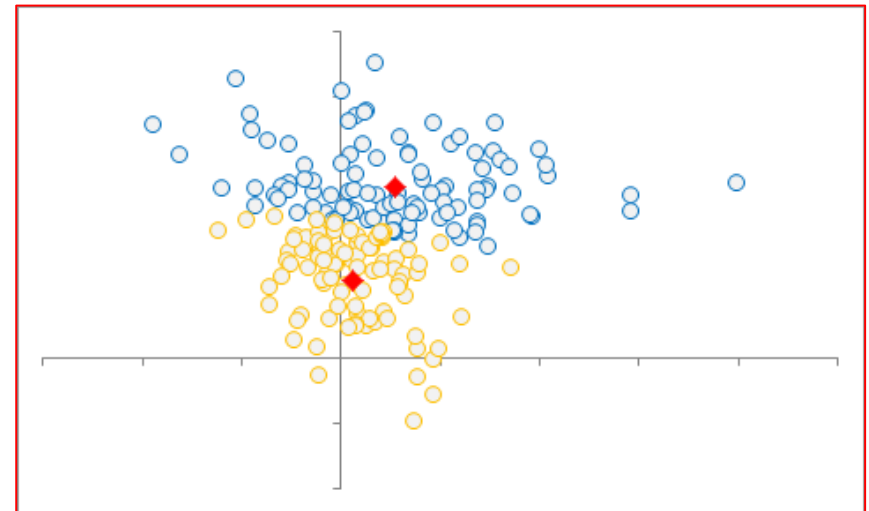
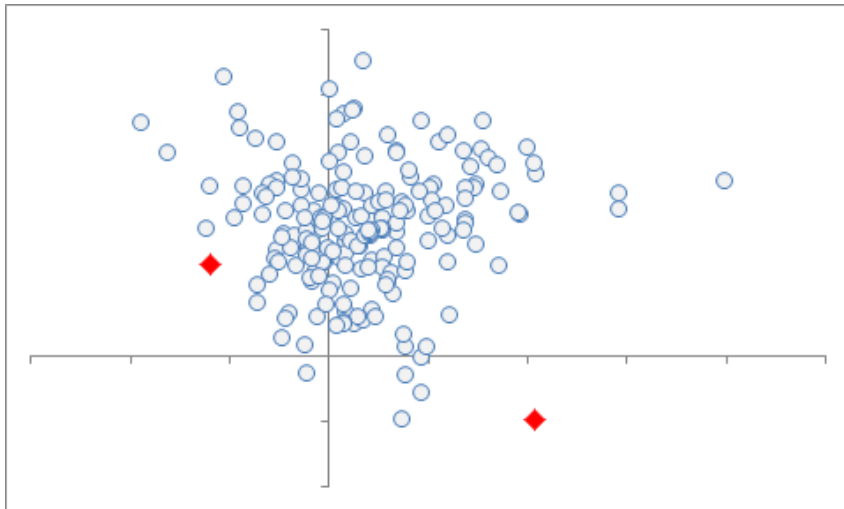
K-means clustering



Unsupervised learning algorithm

- Phân nhóm các điểm dữ liệu thành k cụm (cluster)
- Không biết trước nhãn (label) của từng điểm dữ liệu
- Có 1 hoặc nhiều biến độc lập $x_1, x_2, \dots$
- **KHÔNG** có biến phụ thuộc y

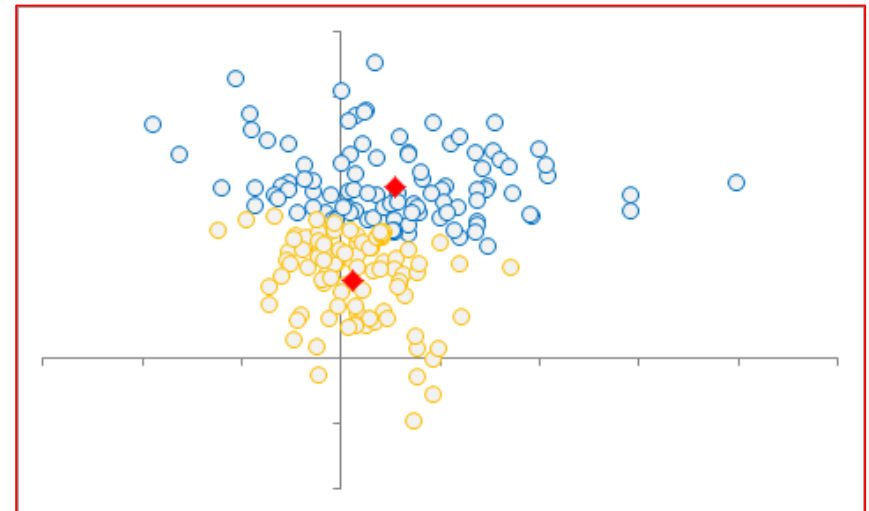
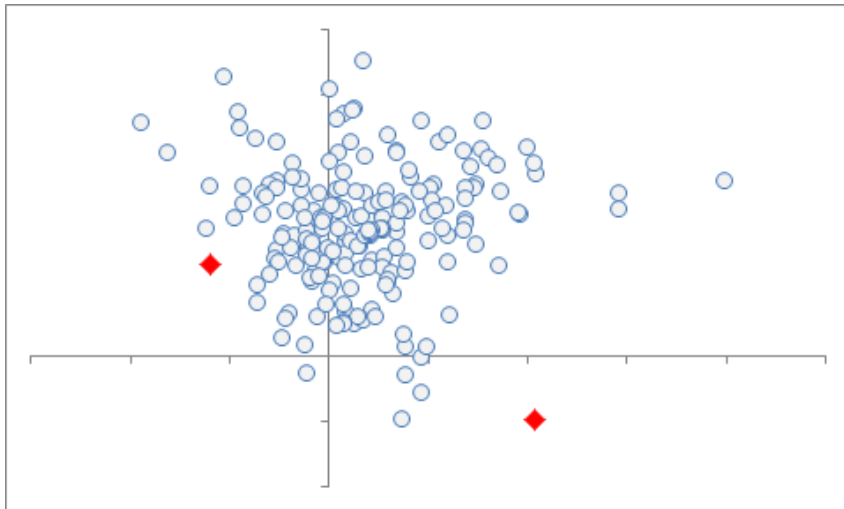
$K=2$



Centroid

- Mỗi cluster có 1 centroid, k cluster sẽ có k centroid
- Các điểm dữ liệu gần với centroid nào nhất thì thuộc cụm của centroid ấy
- Mục tiêu: thuật toán phải tìm ra centroid

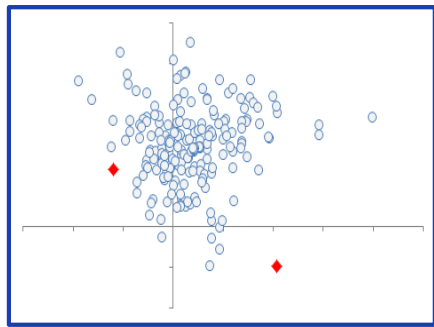
K = 2



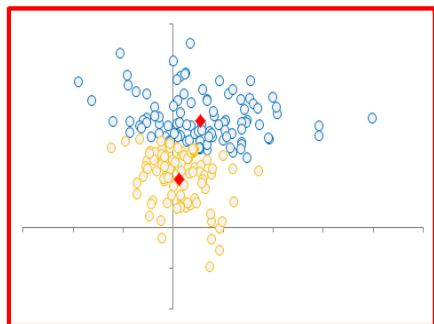
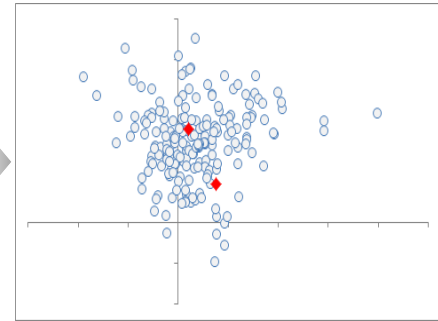
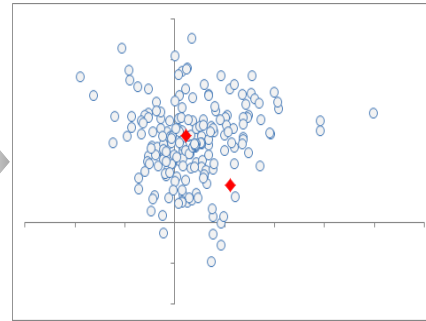
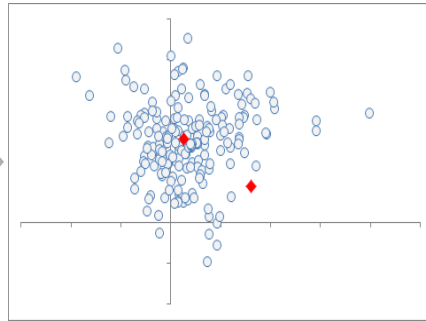
Find the centroids

1. Chọn k điểm bất kỳ trong tập dữ liệu huấn luyện (training set) làm các centroid ban đầu
2. Xếp các điểm dữ liệu vào cụm có centroid gần nó nhất
3. **Cập nhật centroid mới** (được tính bằng trung bình cộng của tất cả các điểm của cụm hiện hành) cho từng cụm
 - Centroid mới có thể không thuộc tập dữ liệu huấn luyện
4. Phân cụm theo centroid mới cập nhật
5. Nếu có bất kỳ 1 điểm dữ liệu nào có thay đổi cụm thì quay lại bước 3. Nếu sự phân cụm theo centroid mới không thay đổi so với centroid cũ thì thuật toán dừng lại

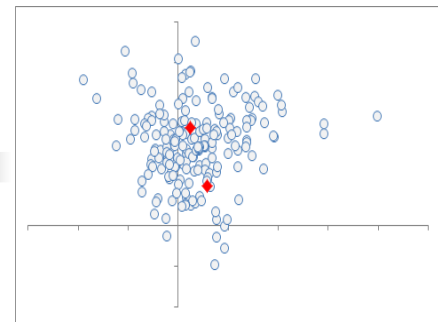
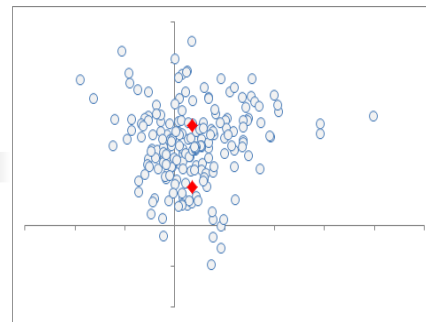
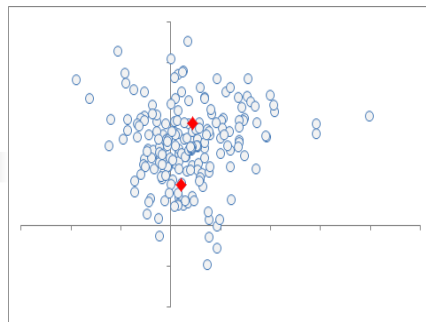
Find the centroids ($k=2$)



Start



Finish



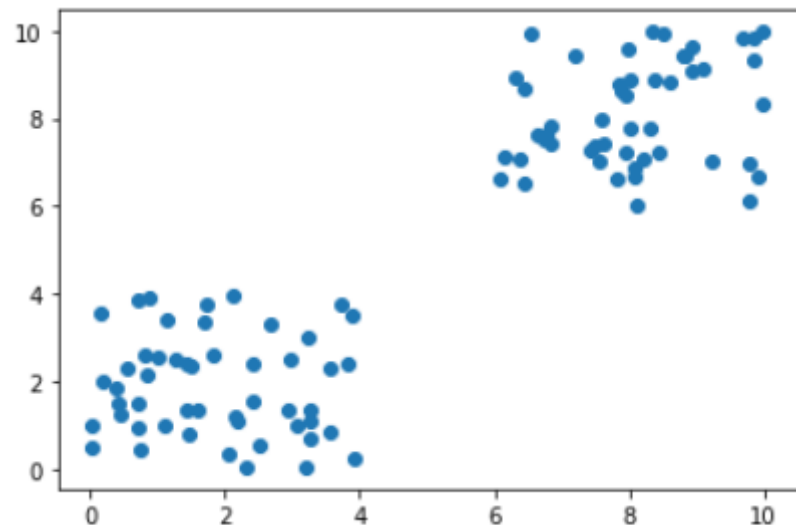
Implement by Python

```
1 def kmeans_init_centroids(X, k):
2     # randomly pick k rows of X as initial centroids
3     return X[np.random.choice(X.shape[0], k, replace=False)]
4
5 def kmeans_assign_labels(X, centroids):
6     # calculate pairwise distances btw data and centroids
7     D = cdist(X, centroids)
8     # return index of the closest centroid
9     return np.argmin(D, axis = 1)
10
11 def has_converged(centroids, new_centroids):
12     # return True if two sets of centroids are the same
13     return (set([tuple(a) for a in centroids]) ==
14            set([tuple(a) for a in new_centroids]))
15
16 def kmeans_update_centroids(X, labels, K):
17     centroids = np.zeros((K, X.shape[1]))
18     for k in range(K):
19         # collect all points that are assigned to the k-th cluster
20         Xk = X[labels == k, :]
21         centroids[k,:] = np.mean(Xk, axis = 0) # then take average
22     return centroids
```

Generate simulated data

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 np.random.seed(100) # Initialize the random seed (100=seed).
5 # np.random.seed()
6 N=50
7 X0 = 4*np.random.random((N,2)) # Return random floats in [0.0, 4.0).
8 print(X0)
9 print('-----')
10 X1 = 4*np.random.random((N,2))+6 # Return random floats in [6.0, 10.0).
11 print(X1)
12 print('-----')
13 X = np.append(X0,X1,axis=0)
14 print(X)
```

```
1 plt.scatter(X[:, 0], X[:, 1])
```



Apply k-means with sklearn

```
1 # Train
2 kmeans = KMeans(n_clusters=4, random_state=123)
3 kmeans.fit(X)
4 print('Centers found by scikit-learn:')
5 print(kmeans.cluster_centers_)
6
7 pred_label = kmeans.predict(X) # print(kmeans.labels_)
8 print(pred_label)
```

Centers found by scikit-learn:

```
[[3.01716303 1.58042434]
```

```
 [7.62075639 7.2473535 ]
```

```
 [0.95695993 2.17874702]
```

```
 [8.62696328 9.30212221]]
```

```
[0 2 2 0 2 0 2 2 0 0 2 0 2 2 0 0 2 2 0 0 0 0 2 0 2 0 2 2 2 2 0 0 0 2 2 2
 0 0 2 0 2 2 0 2 0 2 2 2 2 3 1 3 3 1 1 3 1 1 3 3 1 1 1 1 1 1 3 3 1 1 1 3
 1 1 3 1 1 1 1 3 3 1 3 1 1 3 1 3 3 1 3 3 1 1 3 3 3 1]
```

Predict

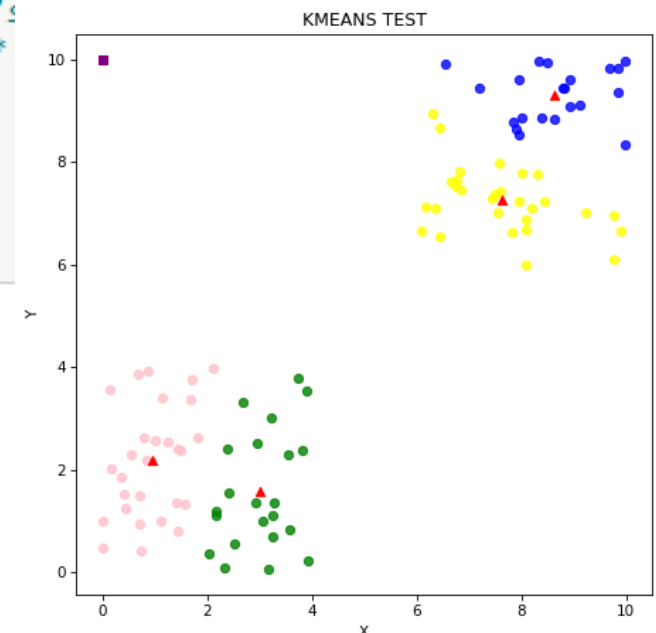
```
1 # A list that contains the learned labels.  
2 learnedLabels = ['Green', 'Yellow', 'Pink', 'Blue']
```

```
1 # Predict  
2 x_test=[[0,10]]  
3 # x_test=[[7,7], [3,2], [2,8]]  
4 # predCluster = kmeans.predict(x_test)  
5 predCluster = kmeans.predict(x_test)[0]  
6 print("Cluster {}: '{}'".format(predCluster, learnedLabels[predCluster]))
```

Cluster 2: 'Pink'

Visualize

```
1 # Visualize
2 x=X[:, 0]
3 y=X[:, 1]
4 plt.figure(figsize=(7, 7), dpi=80)
5 for i in range(x.size):
6     if pred_label[i]==0: color='green'
7     elif pred_label[i]==1: color='yellow'
8     elif pred_label[i]==2: color='pink'
9     else: color='blue'
10    plt.scatter(x[i], y[i], c=color, alpha=0.8)
11
12 plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1], c='red', marker='^')
13
14 # Predict data
15 # plt.scatter([7,3,2], [7,2,8], c='purple', marker='s') #'s'
16 plt.scatter([0], [10], c='purple', marker='s') #'s' '^' '*'
17
18 plt.xlabel('X')
19 plt.ylabel('Y')
20 plt.title('KMEANS TEST')
21 plt.show()
```

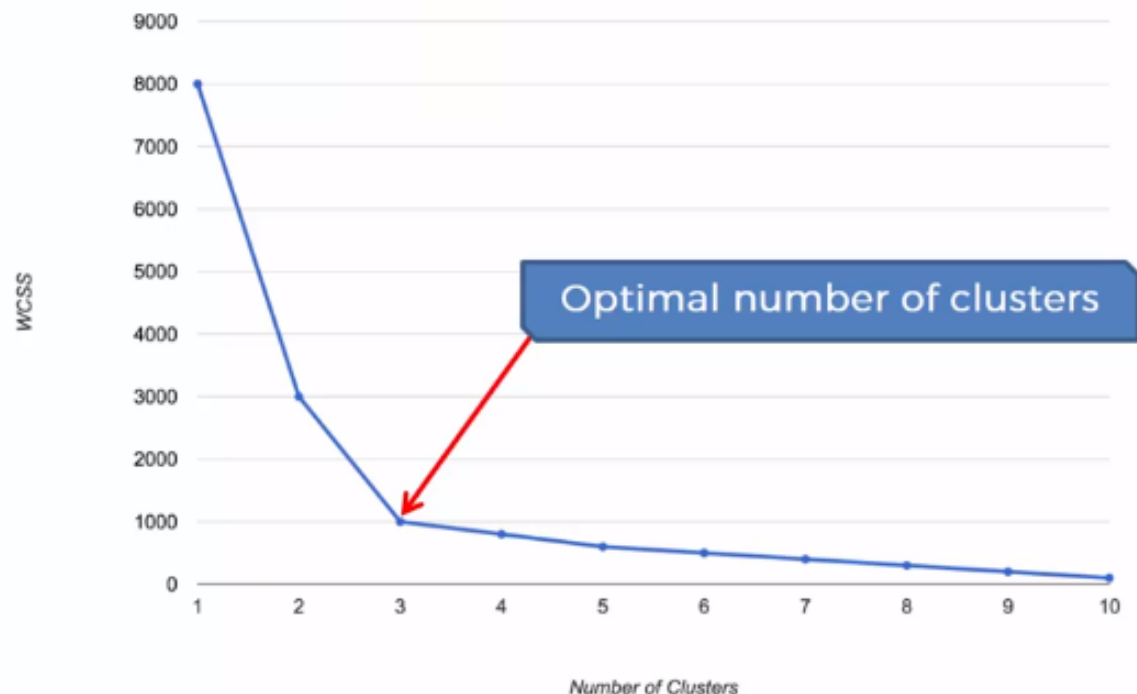


Choosing the right K - The Elbow method

WCSS - Within Cluster Sum of Squares

$$WCSS = \sum_{P_i \text{ in Cluster 1}} \text{distance}(P_i, C_1)^2 + \sum_{P_i \text{ in Cluster 2}} \text{distance}(P_i, C_2)^2 + \sum_{P_i \text{ in Cluster 3}} \text{distance}(P_i, C_3)^2$$

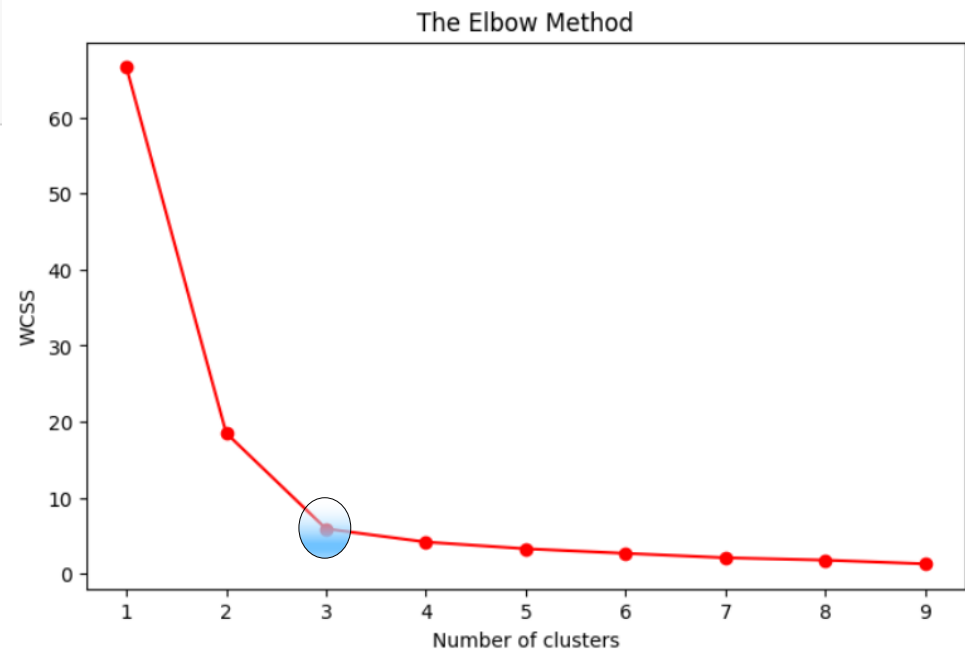
The Elbow Method



The Elbow Example

```
1 # Using the elbow method to find the optimal number of clusters
2 wcss = []
3 for i in range(1, 10):
4     kmeans = KMeans(n_clusters = i, random_state = 100)
5     kmeans.fit(X)
6     # inertia method returns wcss for that model
7     wcss.append(kmeans.inertia_)
```

```
1 plt.figure(figsize=(8,5))
2 plt.plot(range(1, 10), wcss, marker='o', color='red')
3 plt.title('The Elbow Method')
4 plt.xlabel('Number of clusters')
5 plt.ylabel('WCSS')
6 plt.show()
```



Bài tập 1

- Chọn tập X gồm 10 điểm dữ liệu có 2 đặc trưng
- Phân cụm X với $k=2$, $k=3$ theo thuật toán Kmeans
- Vẽ các đồ thị để biểu diễn trực quan qui trình thực hiện

Bài tập 2

- Phân lớp dữ liệu của Bài tập 1 sử dụng thư viện sklearn. So sánh kết quả của 2 Bài tập 1 và 2
- Dự báo phân lớp của 1 điểm dữ liệu mới

Bài tập 3 – phân loại hoa IRIS

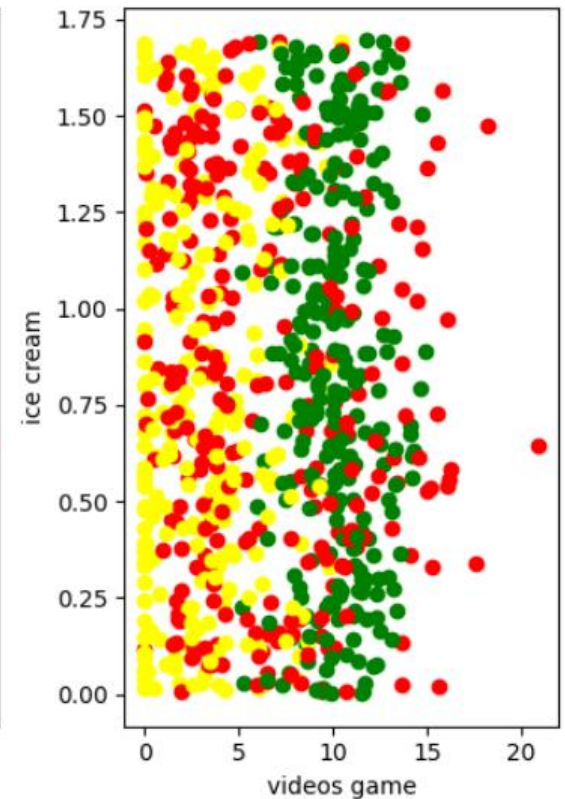
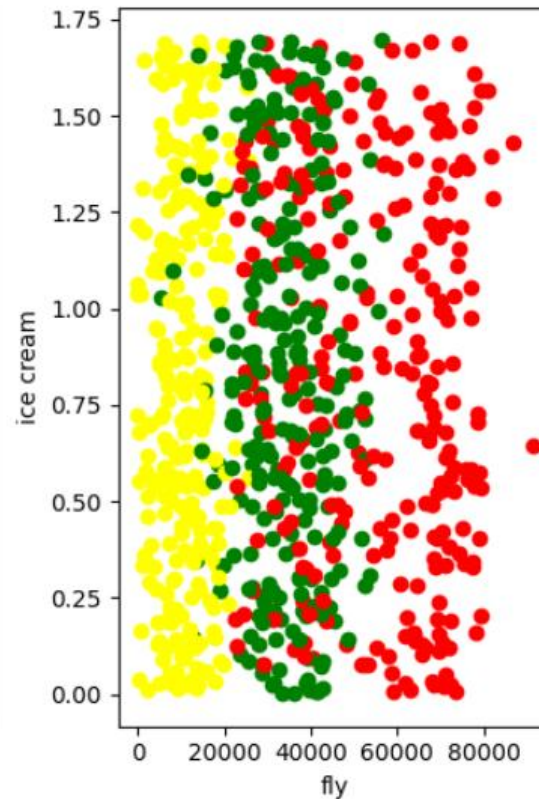
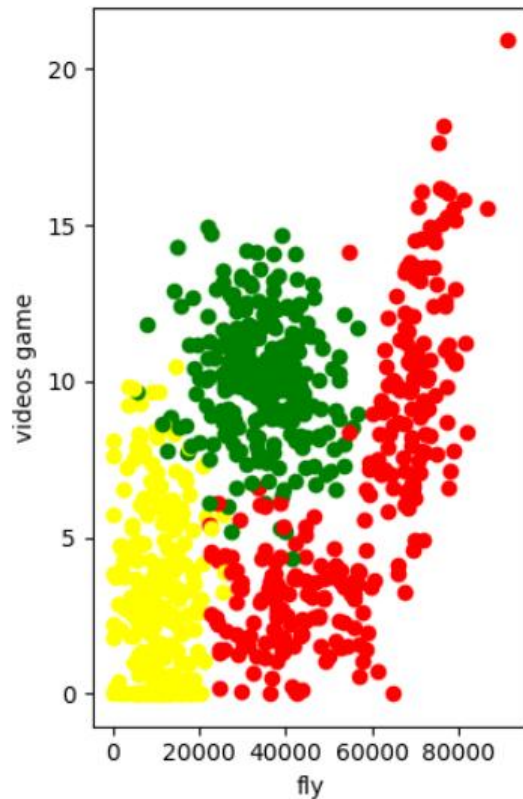
1. Đọc dữ liệu từ seaborn
2. Phân lớp bằng thuật toán Kmeans và vẽ đồ thị biểu diễn sự phân lớp
3. So sánh kết quả dự đoán với kết quả thực. Xuất các điểm dữ liệu dự báo sai

```
import seaborn as sns  
df = sns.load_dataset('iris')
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Bài tập 4 - Dating site

	fly	videos game	ice cream	like
0	40920	8.326976	0.953952	largeDoses
1	14488	7.153469	1.673904	smallDoses
2	26052	1.441871	0.805124	didntLike



Bài tập 4

1. Chọn 2 cột dữ liệu để phân loại (like) theo thuật toán Kmeans với clusters=4
2. Biểu diễn đồ thị phân lớp và hiển thị tâm (centroid) của từng lớp với màu khác và kích thước lớn hơn

Bài tập 5

1. Chọn 2 cột dữ liệu để phân loại (like) theo thuật toán Kmeans với clusters=4
2. Biểu diễn đồ thị phân lớp và hiển thị tâm (centroid) của từng lớp với màu khác và kích thước lớn hơn
3. Tính tỉ lệ dự báo sai
4. Dự báo điểm dữ liệu mới (tự chọn)
5. Dùng phương pháp Elbow để tìm số lượng phân lớp đúng nhất

Bài tập 6

1. Dùng phương pháp Elbow để tìm số lượng phân lớp (k) đúng nhất với dữ liệu đủ 3 cột (fly, videos game và ice cream)
2. Huấn luyện Kmeans với $n_clusters = k$ (câu 1)
3. Kiểm tra lại độ chính xác của thuật toán

End of Document

